Navigation entre les écrans avec NavigationStack

420-249-AH - Programmation mobile iOS

1. Pourquoi utiliser la navigation ?

Une application mobile contient généralement plusieurs écrans.

Exemple :

```
Accueil
   ↓
Produits
   ↓
Détail du produit
```

SwiftUI utilise principalement :

```
NavigationStack
```

pour gérer ce type de navigation.

2. NavigationStack

NavigationStack définit un espace de navigation.

```swift
struct ContentView: View {
    var body: some View {
        NavigationStack {
            Text("Accueil")
        }
    }
}
```

Les écrans affichés à l'intérieur peuvent ensuite ouvrir d'autres vues.

3. Ajouter un titre

```swift
NavigationStack {
    Text("Bienvenue")
        .navigationTitle("Accueil")
}
```

Résultat :

```
Accueil
———————
Bienvenue
```

4. NavigationLink

NavigationLink permet d'ouvrir une autre vue.

```
NavigationLink("Voir le profil") {
    ProfilView()
}
```

Lorsque l'utilisateur appuie sur Voir le profil, SwiftUI affiche ProfilView().

5. Exemple avec deux écrans

Premier écran :

```
struct ContentView: View {
    var body: some View {
        NavigationStack {
            VStack {
                Text("Accueil")
                NavigationLink("Ouvrir le profil") {
                    ProfilView()
                }
            }
            .navigationTitle("Accueil")
        }
    }
}
```

Deuxième écran :

```
struct ProfilView: View {
    var body: some View {
        Text("Profil utilisateur")
            .navigationTitle("Profil")
    }
}
```

6. Bouton de retour

Lorsqu'un écran est ouvert avec NavigationLink, SwiftUI ajoute automatiquement un bouton de retour.

Exemple :

```
< Accueil        Profil
```

Il n'est généralement pas nécessaire de créer manuellement ce bouton.

7. Passer des données à un écran

Une vue peut recevoir des données.

Exemple :

```
struct DetailProduitView: View {
    let nom: String
    let prix: Double

    var body: some View {
        VStack {
            Text(nom)
            Text("\(prix)")
        }
    }
}
```

Navigation :

```
NavigationLink("Voir iPhone") {
    DetailProduitView(
        nom: "iPhone",
        prix: 1200
    )
}
```

8. Exemple avec un modèle

```
struct Produit {
    let nom: String
    let prix: Double
}
```

Vue détail :

```
struct DetailProduitView: View {
    let produit: Produit

    var body: some View {
        VStack {
            Text(produit.nom)
                .font(.title)
            Text("\(produit.prix) $")
        }
    }
}
```

Le modèle Produit regroupe les données, plutôt que de les recevoir séparément.

9. Liste avec navigation

```
let produits = [
    Produit(nom: "iPhone", prix: 1200),
    Produit(nom: "MacBook", prix: 2200),
    Produit(nom: "AirPods", prix: 250)
]
```

Puis :

```
List(produits, id: \.nom) { produit in
    NavigationLink {
        DetailProduitView(
            produit: produit
        )
    } label: {
        Text(produit.nom)
    }
}
```

```
// C'est l'équivalent de :
List(produits, id: { produit in
    produit.nom
}) { produit in
    NavigationLink {
        DetailProduitView(
            produit: produit
        )
    } label: {
        Text(produit.nom)
    }
}
```

Chaque élément peut ouvrir son propre écran de détail.

10. Structure générale

```
NavigationStack
      ↓
    Liste
      ↓
NavigationLink
      ↓
  Vue détail
```

Exemple :

```
Produits
   ↓
iPhone
   ↓
Détail iPhone
```

11. Style du titre

Chaque écran peut avoir son propre titre, par exemple :

```
.navigationTitle("Produits")
// ou
.navigationTitle("Détails")
```

On peut aussi modifier l'affichage du titre avec :

```
.navigationBarTitleDisplayMode(.inline)
```

Exemple :

```
.navigationTitle("Profil")
    .navigationBarTitleDisplayMode(.inline)
```

12. Navigation avec une valeur

Pour les applications plus structurées, on peut naviguer à partir d'une valeur.

Exemple :

```
NavigationLink(
    value: produit
) {
    Text(produit.nom)
}
```

Puis définir l'écran associé :

```
.navigationDestination(
    for: Produit.self
) { produit in
    DetailProduitView(
        produit: produit
    )
}
```

13. Hashable

Pour utiliser NavigationLink(value:), un type doit généralement respecter :

```
Hashable
```

Exemple :

```
struct Produit: Hashable {
    let nom: String
    let prix: Double
}
```

14. Exemple avec navigationDestination

```
struct Produit: Hashable {
    let nom: String
    let prix: Double
}
```

```
struct ProduitsView: View {
    let produits = [
        Produit(nom: "iPhone", prix: 1200),
        Produit(nom: "MacBook", prix: 2200),
        Produit(nom: "AirPods", prix: 250)
    ]

    var body: some View {
        NavigationStack {
            List(produits, id: \.self) { produit in
                NavigationLink(
                    produit.nom,
                    value: produit
                )
            }
            .navigationTitle("Produits")
            .navigationDestination(
                for: Produit.self
            ) { produit in
                DetailProduitView(
                    produit: produit
                )
            }
        }
    }
}
```

15. Navigation simple ou avec valeur ?

Navigation directe :

```
NavigationLink("Profil") {
    ProfilView()
}
```

Appropriée pour :

- Navigation simple
- Écran fixe
- Petite application

Navigation avec valeur :

```
NavigationLink(value: produit)
// avec
.navigationDestination(...)
```

Appropriée pour :

- Listes
- Données dynamiques
- Navigation plus structurée

16. Navigation programmatique

NavigationStack peut aussi gérer une navigation avec un chemin :

```
NavigationStack(path: $path)
```

Exemple :

```
@State private var path: [Produit] = []
```

Puis :

```
path.append(produit)
```

permet d'ouvrir un écran depuis le code.

17. Exemple simple

```
struct ContentView: View {
    @State private var path: [String] = []

    var body: some View {
        NavigationStack(path: $path) {
            Button("Ouvrir Profil") {
                path.append("profil")
            }
            .navigationDestination(
                for: String.self
            ) { destination in
                if destination == "profil" {
                    ProfilView()
                }
            }
        }
    }
}
```

18. Revenir en arrière depuis le code

Une vue peut fermer l'écran actuel avec :

```
@Environment(\.dismiss)
private var dismiss
```

Puis :

```
Button("Fermer") {
    dismiss()
}
```

19. Exemple — Retour depuis le profil

```
struct ProfilView: View {
    @Environment(\.dismiss)
    private var dismiss

    var body: some View {
        VStack {
            Text("Profil")
            Button("Retour") {
                dismiss()
            }
        }
    }
}
```

20. À retenir

Rôle	Outil
Conteneur de navigation	NavigationStack
Navigation vers un écran	NavigationLink
Titre	.navigationTitle()
Navigation basée sur les données	NavigationLink(value:) + .navigationDestination(for:)
Retour programmatique	dismiss()

21. Exemple global

```
NavigationStack
        ↓
  ProduitsView
        ↓
  NavigationLink
        ↓
DetailProduitView
        ↓
Bouton retour automatique
```

22. Exercice — Catalogue de véhicules

Créer :

```
struct Vehicule: Hashable
```

avec :

```
nom
marque
prix
```

Créer ensuite :

```
VehiculesView
```

qui affiche une liste de plusieurs véhicules. Chaque véhicule doit être accessible avec :

```
NavigationLink
```

Créer :

```
DetailVehiculeView
```

qui affiche :

```
nom
marque
prix
```

Utiliser :

```
NavigationStack
NavigationLink
navigationDestination
navigationTitle
```

23. Exercice — Retour manuel depuis le détail

Ajouter dans DetailVehiculeView un bouton permettant de revenir manuellement avec :

```
dismiss()
```